CSS 217 Lecture Handout: Beyond Design Patterns

Practical Java Optimization, the JVM, and Clean Code Realities

Instructor - Sufyan bin Uzayr

Section 1: The JVM Reality of Abstraction

1. The Core Philosophy

> *"Premature optimization is the root of all evil."* — Donald Knuth
> **Corollary:** *"Premature abstraction is the root of technical debt."*

Design patterns (Proxy, Decorator, Adapter, Bridge, Strategy) decouple architecture, but every layer of abstraction carries a runtime cost. Design for **anticipated, multi-axis change**, not speculative flexibility. If a behavior will never swap or mutate at runtime, a plain class or method is almost always superior to an interface with one implementation.

---

2. How the JVM Executes Your Patterns

When you write nested decorators or dynamic strategies, the JVM does not see "clean architecture"—it sees pointers and dispatch tables.

```
None
[ CPU Core ] <---> [ L1/L2 Cache ] <---> [ L3 Cache ] <---> [ Main Memory (Heap) ]
                                                                      |
      Pointer Hops (Cache Misses):                                    ▼
      Decorator A ──(ptr)──> Decorator B ──(ptr)──> Proxy ──(ptr)──> Target Object
```

A. Pointer Chasing & Cache Lines

- Modern CPUs fetch data into ultra-fast caches in **64-byte lines**.
- Contiguous memory (e.g., primitive arrays `int[]`) allows sequential prefetching.

- Deeply nested patterns (**Decorator**, **Bridge**, **Adapter**) store references to other heap objects scattered across memory. Following these pointers (`this.inner.doTask()`) forces CPU cache misses, stalling the processor.

B. JIT Inlining & Call Site Polymorphism

The HotSpot JIT (Just-In-Time) compiler makes Java fast primarily via **method inlining** (substituting the method call directly with its body).

- **Monomorphic Dispatch (1 implementation):** HotSpot detects the concrete target and directly inlines it. Zero dispatch overhead.
- **Bimorphic Dispatch (2 implementations):** Handled efficiently using a conditional branch.
- **Megamorphic Dispatch (≥ 3 implementations):** Inlining fails. The JVM must perform a dynamic table lookup (`invokevirtual` via vtable) on every invocation.
- *Takeaway:* A dynamic **Strategy** pattern used across hundreds of implementations at a single call site eliminates JIT inlining opportunities.

C. Heap Allocation & GC Churn

- Creating transient wrappers (e.g., throwing a new `Adapter` or `LoggingProxy` around an object inside a tight loop) rapidly fills Eden space.
- High allocation rates trigger frequent Young Generation Garbage Collection pauses (Stop-The-World events).

---

3. Pattern Pitfalls: What to Watch For

Pattern	Intended Benefit	Runtime & Maintenance Trap
Decorator	Open-ended compositional behavior	Obscured stack traces; 6+ pointer dereferences per call; wrapper sprawl.
Strategy	Swappable runtime algorithms	Megamorphic call sites prevent inlining; often over-engineered for static logic.
Proxy	Lazy loading, access control, caching	Hidden I/O overhead; dynamic proxies (`java.lang.reflect.Proxy`) incur reflection cost.

Pattern	Intended Benefit	Runtime & Maintenance Trap
Adapter	Unifies incompatible APIs	Memory allocation churn if created per-call instead of reused; object indirection.
Bridge	Decouples abstraction from implementation	Double-dispatch overhead; complex class hierarchy for simple domain problems.

Section 2: Pragmatic Java Optimization & Clean Code

4. Four Everyday Java Performance Anti-Patterns

Trap 1: Autoboxing in Generic Interfaces

```java
// ❌ BAD: Allocates a new java.lang.Integer on the heap for every invocation
public interface ScoreCalculator<T> {
    Integer compute(T record);
}

// ✅ GOOD: Zero heap allocation; operates purely in registers/stack
public interface ScoreCalculator<T> {
    int compute(T record); // Or use java.util.function.ToIntFunction<T>
}
```

Trap 2: Under-sized Collection Growth

When resizing an `ArrayList`, Java creates a new internal array ($\approx 1.5 \times$ larger) and copies every element via `System.arraycopy`.

```java
// ❌ BAD: Resizes repeatedly if reading 100,000 items
List<String> records = new ArrayList<>();

// ✅ GOOD: Allocates contiguous block upfront; zero resizing copies
List<String> records = new ArrayList<>(expectedSize);
```

Trap 3: String Concatenation in Loops

```java
// ❌ BAD: O(n²) time; creates intermediate String and StringBuilder objects per iteration
String log = "";
for (String token : tokens) { log += token + ", "; }

// ✅ GOOD: Amortized O(n); single backing buffer
StringBuilder log = new StringBuilder(tokens.size() * 16);
for (String token : tokens) { log.append(token).append(", "); }
```

Trap 4: Misapplying Java Streams to Hot Loops

- Java Streams are fantastic for business logic readability, but each stage creates pipeline objects and lambda captures.
- In high-throughput, low-latency loops (processing millions of elements/sec), classic counted `for` loops remain faster and generate zero GC garbage.

---

5. The Engineering Mindset: How to Optimize

1. **Write Clean, Idiomatic Code First:** Never optimize based on intuition. Human intuition regarding modern JIT compilers is notoriously inaccurate.
2. **Profile Before Touching Code:**
    - Use **VisualVM** (GUI, bundled with JDK/available free) for thread and heap snapshots.
    - Use **async-profiler** for low-overhead production flamegraphs of CPU and allocation hot paths.
3. **Respect the 90/10 Rule:** 90% of execution time is almost always spent in 10% of your code (usually database access, network I/O, or a nested loop). Optimizing the other 90% is wasted engineering capital.

---

6. The "Before You Abstract" Checklist

Before creating a new Interface, Strategy, or Decorator, ask:

- ☐ Does this concrete class have **more than one** actual implementation right now?
- ☐ Will this branch logic change dynamically at runtime, or is it fixed at compile time?
- ☐ Does an existing standard library utility (`Function<T, R>`, `Predicate<T>`, `Supplier<T>`) solve this without a custom interface?
- ☐ Will this design obfuscate the stack trace for someone debugging at 3:00 AM?

- [ ] Is this abstraction solving a real requirement today, or an imaginary requirement for next year?